

1. Project Guideline - Getting Started

- 1. Project Guideline - Getting Started
 - 1.1. What to learn: Prerequisites
 - 1.1.1. Python
 - 1.1.2. Code environment & Notebook
 - 1.1.2.1. Environments
 - 1.1.2.2. Notebook
 - 1.1.2.3. Programming Application
 - 1.1.3. Data Wrangling & Machine Learning
 - 1.1.4. Computer Vision
 - 1.1.4.1. OpenCV
 - 1.1.4.2. Deep learning: TensorFlow, Pytorch and so on
 - 1.2. General preparations
 - 1.2.1. Learn how to write your report, and prepare them
 - 1.2.1.1. Structure guideline for a research report
 - 1.2.1.2. Examples
 - 1.2.2. Learn how to do reports on Latex
 - 1.2.3. To-dos after having picked your topic
 - 1.2.3.1. Literature review - Defining your topic
 - 1.2.3.2. Code replication

1.1. What to learn: Prerequisites

Before you head on, this is what a full graph of the tech stacks behind an AI product:

```

Programming
├─ Python
│   ├── Data: numpy, pandas
│   ├── ML: scikit-learn
│   ├── DL: tensorflow, pytorch
│   └─ Products: APIs, dashboards, batch jobs
│       └─ Full-stack (FastAPI/Django + React; DB; Docker/K8s)
├─ C++ (performance & production enablement)
│   ├── Core: Modern C++ (17/20), STL, RAII
│   ├── Speed on CPU: OpenMP (simple parallel loops), oneTBB (pipelines), SIMD
│   ├── GPU path: CUDA (NVIDIA) → cuBLAS/cuDNN; alt: SYCL(oneAPI) or ROCm
│   ├── Python interop: pybind11 (wrap hotspots), PyTorch/TensorFlow custom op
│   └─ Serving: ONNX Runtime / TensorRT (low-latency inference), Triton Infer
└─ Full-stack Programming
    ├── Backend: FastAPI, Flask, Django, Node
    ├── Frontend: React/Vue/Svelte
    ├── Databases: Postgres/MySQL, Redis, MongoDB
    ├── DevOps: Docker, K8s, CI/CD, Observability
    └─ Deploy: Cloud/Edge, model serving (TorchServe, TF Serving, Triton)
        └─ ML Products

```



You need to try out things to really know what you like, and what makes you feel good.

1.1.1. Python

At the most basic, to be able to do Machine Learning, you need to know, in this order:

- Basic Python programming
- Data wrangling
- Machine Learning

If you do not know Python yet, take this 3-part course on [Grok Academy](#) (2 days should be sufficient). Then look into [Python Documentation](#) when you need something more advanced. FreeCodeCamp Youtube channel is also a good option.

1.1.2. Code environment & Notebook

1.1.2.1. Environments

It is very important to know about how you set up your code environment, which maintains the libraries you are using in each project, configurations and so on. The 1st step to make your project well-organised.

The most simple way is using venv. The quickest and most common way is to install [Anaconda](#).

1.1.2.2. Notebook

Coding on notebook (.ipynb) is the most common way used by scientists. It allows you to blend between markdown texts and code segments. So it enables you to describe what you do, and show the results running each separate code segment - really useful for debugging.

The most commonly used Notebooks are Jupyter (local) and Google Colab (online). Anaconda should also have Jupyter Notebook available as an installable software.

1.1.2.3. Programming Application

PyCharm or VSCode are common choices. VSCode is recommended as it adapts to other programming languages as well!

1.1.3. Data Wrangling & Machine Learning

After getting used to Python basics, you can start learning how to use the most common libraries.

Python is best known for data processing. Its most commonly-used library is `numpy`, and `pandas`.

After that, you are ready to learn basic Machine Learning. `scikit-learn` is the most commonly-used package, and as you learn the basic ML concepts and models, it should have most of them covered in the code and you can call them directly.

Note that since the time is limited, don't try to grasp everything. For now, just try to grasp the 'minimum required knowledge' that I will be mentioning below:

- Training/Testing data
- The model type definitions such as supervised, unsupervised, classification, regression,...
- Metrics: Accuracy, Precision, Recall, F1-Scores
- (Briefly) How some basic model works: Naive Bayes, Linear Regression, Decision Tree, KNN, Logistic Regression,...
- How to do ML on Python

Where to learn:

- The theory: [University of Melbourne's COMP30027 Lecture Notes](#)
- The code + Exercises: [University of Melbourne's COMP90049 Tutorial Notebooks](#) - these are my modified version of the exercises notebooks that I used to taught Masters Students in 2023, for the subject Introduction to Machine Learning.
- What's next: Of course the course cannot cover everything, so learn how to read library documentations when you have any questions or want to learn new libraries/models.

1.1.4. Computer Vision

From ML, you dip your first foot into the world of AI. There are Big Data, NLP, Computer Vision and Reinforcement Learning - the 4 main AI branches. We will delve into Computer Vision - where you will attempt to extract information from image/video inputs and return something out of it.

Note that since the time is limited, don't try to grasp everything. For now, just try to grasp the 'minimum required knowledge' that I will be mentioning below:

- Pinhole camera model and their property matrices (intrinsic, extrinsic)
- Convolution Neural Networks mechanism
- Different deep learning structures
- How to implement OpenCV, and deep learning

Resource links:

- The theory: [University of Melbourne's COMP90086 Lecture Notes](#)
- The code + Exercises: [University of Melbourne's COMP90086 Tutorial Notebooks](#) - these are my modified version of the exercises notebooks that I used to taught Masters Students in 2023, for the subject Computer Vision.

1.1.4.1. OpenCV

It's like Photoshop but on code, and it has many more image functionalities. Its documentation is brought all over from the C++ implementation, so it might be a bit hard to read. Consult AIs like ChatGPT and Cursor if needed.

1.1.4.2. Deep learning: TensorFlow, Pytorch and so on

Of course, to be able to know how to do deep learning, either one of TensorFlow or Pytorch is a must. While TensorFlow is taught in the resources above, you can also look for places to self-learn Pytorch - the more commonly used library (and harder to use). After this you can start to discover more deep learning packages across GitHub such as OpenMM and the likes.

1.2. General preparations

1.2.1. Learn how to write your report, and prepare them

In case you wonder, your report is expected to be most aligned with the Research Report type.

1.2.1.1. Structure guideline for a research report

1. **Title Page** (not counted in word count)

- Title of your research
- Your name, course, university, date
- Sometimes student ID, supervisor, department

2. **Abstract (150–250 words)**

- Short summary of the research: purpose, methods, results, conclusion.
- Write it last, even though it comes first.
- Keep it concise: one sentence each for *background*, *aim*, *methods*, *findings*, *implications*.

3. **Introduction (400–700 words)**

- **Context:** Why is this topic important? (background, significance)
 - **Problem statement:** What gap/question does your research address?
 - **Objectives / Research questions / Hypotheses**
 - **Scope & limitations:** What you will and will not cover
 - Brief **overview of the structure** of your paper
-

4. Literature Review (800–1,200 words)

- Summarize **key theories, models, prior studies**
 - Compare and contrast findings
 - Identify **gaps or controversies**
 - Show how your research builds on or differs from existing knowledge
 - Organize logically (thematic, chronological, or methodological)
-

5. Methodology (500–800 words)

- **Research design:** qualitative, quantitative, experimental, case study, etc.
 - **Data collection methods:** surveys, experiments, interviews, secondary data
 - **Sample:** who/what you studied, and why
 - **Tools/instruments** used (software, equipment, tests)
 - **Procedure:** step-by-step process
 - **Ethical considerations:** consent, confidentiality, approvals
 - Justify why this method is appropriate
-

6. Results / Findings (600–1,000 words)

- Present your findings clearly and neutrally (no interpretation yet)
 - Use **tables, graphs, charts, figures** where appropriate
 - Organize by themes, variables, or research questions
 - Highlight key patterns, trends, and significant results
-

7. Discussion (600–1,000 words)

- Interpret the results in relation to your research questions
 - Compare with literature review: do your results confirm or challenge prior work?
 - Discuss implications (theoretical, practical, policy)
 - Acknowledge limitations of your study
 - Suggest areas for future research
-

8. Conclusion (300–500 words)

- Summarize key findings
 - Re-state how you answered your research question(s)
 - Highlight contributions to knowledge / practice
 - Leave the reader with a clear final message
-

9. References / Bibliography

- Correct academic referencing style (APA, MLA, Chicago, etc.)
 - Every in-text citation must appear here
-

10. **Appendices** (optional, not in word count)

- Raw data, survey instruments, interview guides, extra charts/tables

Word Count Balance (for 3,000–5,000 words)

- Introduction: ~10–15%
- Literature Review: ~20–25%
- Methodology: ~15–20%
- Results: ~20%
- Discussion: ~20%
- Conclusion: ~5–10%

Tips

- Write *clearly and formally* (avoid casual language).
- Use **headings & subheadings** to structure.
- Keep paragraphs focused (1 idea per paragraph).
- Always link back to your research objectives.
- Proofread for clarity and flow.

1.2.1.2. Examples

They are all similar in terms of requiring well-structured, well-logic explanation in how you do things. Just different purposes:

- [Research report - Journal/Conference Submission - Short](#) **this is the closest of what expected of yours**
- [Project Report - Short](#)
- [Technical Report - Long](#)
- Look for papers from CVPR, ECCV, ICCV, Springer LNCS formats,...

1.2.2. Learn how to do reports on Latex

Latex == better representation. You will get better praised by the academic people if you use this for your work.

The main thing about Latex is the ability to type mathematical equations, and fast cite/reference system. Know these 2 and you already knew 80% of Latex. It is not too hard to make a Latex report!

- Sign up on [Overleaf](#)
- Pick a sample project with the format you like
- Try to tweak them and make them your report
- Any questions? Type on Google: Latex [your problem]

1.2.3. To-dos after having picked your topic

1.2.3.1. Literature review - Defining your topic

- Look for papers that are related to your topic. Ensure that you found credible sources (CVPR, AAAI, Springer, Elsevir, high-ranking university researchers,...), ~~not some random Indian nonsense or 'tu dai bo tuc' level of bullshit.~~
- Next step is note the important details that you have found to a compilation note, and remember to mention the source.

Example: 'info number 1: Algorithm A has effect B - Paper XXX, YYY'

TIP: Use doi2bib to get the BiBTeX of your reference paper. That way you can insert them to latex much quicker as well when citing.

1.2.3.2. Code replication

Now, try and set up the given resources/reproduce the GitHub code so that your code can run the basics. That way, you can just quickly intergrate upon them when implementing/improving your algorithms